Python-Based STEM

Doing Math with Python

Course Overview

This is a non-graded course made up of eight 70-minute sessions, for about 9 hours and 20 minutes of learning across approximately eight months. Sessions may be several weeks apart. Sessions 2–8 begin with a short re-entry activity so that you can recover useful ideas without needing to remember every detail from the previous session.

We will work in Google Colab, a browser-based notebook where written explanations, Python code, and results can appear together. You do not need previous programming experience, a local Python installation, or software-development tools.

What This Course Is About

The course begins with mathematical questions, not lists of programming commands. We will use Python when it helps us repeat a calculation, search many cases, make a pattern visible, test an idea, or explore variation more effectively than we reasonably could by hand.

The central principle is simple:

> **Python is a tool for doing mathematics; mathematical investigation is the purpose.**

The central learning goal is that you become increasingly able to:

> **ask or interpret a mathematical question, adapt computation to investigate it, and explain what the resulting evidence does—and does not—support.**

The Eight-Session Journey

Each session gives computation a different mathematical role. These roles form a journey rather than a sequence of isolated Python topics.

Session	Computational Role	Mathematical Investigation	Big Idea
1	**Search**	With a fixed perimeter, which whole-number rectangle has the greatest area?	A systematic search can reveal a maximum and symmetry.
2	**Generate**	What happens when we add consecutive odd numbers?	Generated evidence reveals square-number structure and the importance of where a pattern starts.
3	**Visualize and Compare**	How do additive and multiplicative growth separate over time?	Tables and graphs show when similar-looking rules begin to behave very differently.
4	**Test**	In a bounded Collatz investigation using a simple odd/even number rule, do larger starting numbers always take more steps to reach 1?	A counterexample can disprove an "always" claim, while many successful tests are still not universal proof.
5	**Model**	When does a simplified cooling model reach a target temperature?	Model results depend on the rule, parameters, and assumptions we choose.
6	**Analyze**	Can datasets with the same summaries still have different structures?	Mean and range are useful, but distributions can reveal what summaries hide.
7	**Simulate**	How do 30 fair-coin experiments of 20 trials compare with 30 experiments of 500 trials?	Larger random experiments tend to give more stable proportions, but randomness remains.
8	**Adapt**	For rectangles with area at least 40, which whole-number dimensions have the least perimeter?	A familiar search can be adapted to a reversed problem, with conclusions limited by the evidence searched.

The full progression is:

> **Search → Generate → Visualize and Compare → Test → Model → Analyze → Simulate → Adapt**

In everyday language, you will use computation to look systematically, produce useful cases, make differences visible, challenge claims, represent a situation, study data, investigate chance, and transfer a familiar method to a changed question.

What You Will Learn to Do

Across the course, you will practice how to:

- make a prediction before an important result is revealed;
- calculate a manageable case and translate its mathematics into code;
- use Python to handle repetition, scale, systematic search, or random trials;
- read output as mathematical information rather than treating a successful run as the finish line;
- notice patterns, make conjectures, and test them with meaningful new cases;
- compare numerical, tabular, and visual evidence;
- revise a claim when new evidence challenges or qualifies it;
- distinguish evidence from proof; and
- explain what a result supports and what remains uncertain or outside the investigation.

How We Will Learn

Many investigations follow a recurring path:

> **Question → Predict → Try a manageable case → Translate the mathematics → Identify repetition → Let Python automate → Interpret evidence → Make a conjecture → Test a new case → Refine → Transfer**

Not every session will use every step in exactly this order. The important habit is to connect the mathematical question, the computation, the evidence, and the conclusion.

Your independence with Python will also grow through four kinds of work:

- **See:** encounter a code structure and begin to interpret what it does mathematically.
- **Use:** run supplied code for a purpose and explain its output.
- **Modify:** change a meaningful value, expression, condition, or representation.
- **Construct:** write an appropriate small piece of code in a new situation.

You may be ready to construct one kind of code while still using supplied code for another. Using supplied code can still involve serious mathematical thinking. You will not be asked to write large programs from a blank page.

Python We Will Use

The course uses a small, recurring toolkit:

- arithmetic, variables, and `print()`;
- comparisons and supplied `if`/`else` decisions;
- `for` and `range()` for repetition;
- lists, counters, and running totals for keeping track of results;
- a few supplied Python tools for storing, counting, totaling, and comparing results;
- a simple supplied random choice for coin simulations; and
- supplied graphing code when a visual representation helps answer the mathematical question.

New code appears only when it adds mathematical power. You will often receive the surrounding structure and complete only the mathematically important part.

What You Are Not Expected to Master

This is not a compressed software-development course. You are not expected to master advanced Python, write major programs independently, define functions as a central course goal, use object-oriented programming, manage packages or external data, build websites, use Git or GitHub, work from a command line, or create advanced plots.

Typing speed, memorizing syntax, and producing polished software are not course goals. The purpose is to use a manageable amount of code to think more deeply about mathematics.

How Progress Will Be Shown

There are no letter grades, percentages, or points. You will show growth through your work during the investigations: predictions, hand calculations, purposeful code changes, interpreted results, selected evidence, revised claims, stated limitations, short reflections, and mathematical discussion.

Successful work does not require every first prediction to be correct, every line of code to run immediately, the fastest completion, working without hints, or a perfect final explanation. Progress is visible when your reasoning becomes more deliberate: you connect code to mathematics, use output as evidence, and make claims that match what the investigation can justify.

Results, Evidence, Claims, and Limitations

> **Result → Evidence → Claim → Limitation**

These four ideas will anchor the whole course:

- A **result** is what a calculation, search, graph, model, analysis, or simulation produces.
- **Evidence** is a result or group of results that is relevant to a mathematical question.
- A **claim** is a conclusion supported by the evidence.
- A **limitation** identifies a boundary: what was assumed, what was not tested, or what the evidence does not establish.

For example, a bounded search may identify the best rectangle among all cases the program checked. That is strong evidence about the searched range, but it does not automatically prove a statement about every possible rectangle. In a model, Python accurately calculates the consequences of the supplied rule, but the real-world conclusion still depends on whether the assumptions are reasonable. In a simulation, a collection of trials can reveal a tendency without guaranteeing the result of the next experiment.

Working With a Partner

Collaboration usually follows this pattern:

individual thinking → paired investigation → individual explanation

You will often make your own prediction, work with a partner to calculate, debug, run, or compare, and then write your own conclusion. Partners should share control of the mathematical and computational thinking. We will not use permanent “coder” and “recorder” roles; both people should be able to explain what changed, what the output means, and why the evidence matters.

If a device or notebook problem slows you down, pairing is also a normal fallback so that the mathematics can continue.

Mistakes, Debugging, and Revision

An unexpected result is not something to hide. It may reveal a mistaken prediction, a mathematical relationship we overlooked, or a code change that does not mean what we intended. Debugging means comparing the question, the mathematics, the code, and the output until they agree.

When something fails, first ask what each important quantity represents and what the changed line was supposed to do. Then use error messages, nearby examples, partner reasoning, and teacher support. Revising a conjecture after a counterexample and revising code after an error are both forms of thoughtful work.

Attendance and Long Gaps Between Sessions

Because meetings may be separated by several weeks, forgetting some details is expected. Sessions 2–8 begin with a brief retrieval bridge that brings back the few ideas needed for the new investigation and revisits important computational structures. You are not expected to memorize every Python command between sessions. These openings are for rebuilding connections, not judging memory.

Regular participation gives you the fullest experience of the progression. If you miss a session, return ready to use the re-entry activity, the notebook, a partner, and teacher support to reconnect with the course's recurring methods.

Materials

For each session, bring or have access to:

- a computer that can open Google Colab in a web browser;
- reliable internet access as provided by the program;
- your course notebook link or other access instructions supplied by the teacher; and
- something for brief handwritten calculations or notes, if requested.

No additional or paid programming software is required.

Classroom Expectations

Our shared expectations support serious investigation without turning the course into a rule sheet:

- make a prediction or attempt before seeking an answer;
- explain mathematical meaning, not only which buttons or lines you used;
- share control of the thinking and the computer when working with a partner;
- listen to claims that differ from yours and ask what evidence supports them;
- do not erase surprising results merely because they differ from expectations;
- when code fails, first check the mathematics and the intended change; and
- use class time for the investigation rather than unrelated browsing.

Use of AI Tools

You may use AI tools only when the teacher or program permits them. When allowed, AI may help you interpret an error, ask a clarifying question, or consider an approach, but it should support rather than replace your mathematical thinking. You remain responsible for explaining what the code does, why the evidence matters, and what claim is justified.

By the End of the Course

Course completion is based on participating across the investigations and engaging with the mathematical reasoning, not on grades. By the end, your work should show increasing ability to ask or interpret a mathematical question, use computation as a tool, and explain what the resulting evidence supports.

Python can perform repetitive computational work quickly. Mathematics still requires human judgment about the question, the relevant evidence, the justified claim, and the limitations of the conclusion.